BIM304 - Computer Algorithm Design

Assignment 1: Divide-and-Conquer Skyline Problem

Objective. Develop a Java application that recursively finds the skyline of a set of buildings using the divide-and-conquer strategy. Your program must output the critical points that describe the visible outer silhouette of the buildings.

Important. There is at least one building in the input. Every building has three integer values: left coordinate, right coordinate, and height. The right coordinate is always greater than the left coordinate, and the height is positive.

Example

If the input is:

```
5
1 3 4 -> Building: Left: 1, Right: 3, Height: 4
2 5 6 -> Building: Left: 2, Right: 5, Height: 6
7 9 3 -> Building: Left: 7, Right: 9, Height: 3
8 11 5 -> Building: Left: 8, Right: 11, Height: 5
10 12 2 -> Building: Left: 10, Right: 12, Height: 2
```

then the output should be:

```
(1, 4) (2, 6) (5, 0) (7, 3) (8, 5) (11, 2) (12, 0)
```

Explanation. Each pair (x, h) means that the skyline height becomes h starting from x. The final point should return the height to 0. Consecutive skyline points with the same height must not be printed.

Details

- There are 5 input files. The smallest file has 10 buildings, while the largest file has 100,000 buildings.
- The first line specifies the number of buildings in the input data.
- Each subsequent line contains the left coordinate, right coordinate, and height of a building, separated by a single space.
- The output must show the skyline critical points in increasing order of x-coordinate, using the exact format shown in the example.
- Your algorithm must fit into the divide-and-conquer strategy.
- The expected running time is Theta(n log n), where n is the number of buildings.
- Brute-force coordinate scanning and sweep-line solutions using a heap or priority queue are not allowed for this assignment.
- You may use ArrayList, BufferedReader, and StringTokenizer. You must not use Arrays.sort or Collections.sort; you are expected to complete the provided MergeSort.java file.

Input Files

File name	Content
small.input	contains 10 buildings
test_100.input	contains 100 buildings
test_1000.input	contains 1,000 buildings
test_10000.input	contains 10,000 buildings
test_100000.input	contains 100,000 buildings

Input Format

```
n
left_1 right_1 height_1
left_2 right_2 height_2
...
left_n right_n height_n
```

Output Format

```
(x1, h1) (x2, h2) ... (xk, hk)
```

- x-values must be printed in increasing order.
- The last skyline point must have height 0.
- If two or more skyline changes happen at the same x-coordinate, your output must contain only the final visible height for that x-coordinate.
- If two consecutive critical points have the same height, the second one is redundant and must not be printed.

Provided Java Files

File	Instruction
Building.java	Do nothing. This class stores one building.
SkylinePoint.java	Do nothing. This class stores one skyline critical point.
TestSkyline.java	Do nothing. This class reads the input file, calls your methods, and prints the result.
MergeSort.java	Write the required codes for the methods left blank. Never change the method names or file name.
FindSkyline.java	Write the required codes for the methods left blank. Never change the method names or file name.

Files You Need to Upload

- MergeSort.java
- FindSkyline.java

Required Algorithmic Structure

1. Sort the buildings by their left coordinate using your MergeSort.java implementation.

2. Use divide-and-conquer in FindSkyline.java. Divide the building array into two halves.

3. Recursively compute the skyline of the left half and the skyline of the right half.

4. Merge the two skylines by scanning their critical points and tracking the current height of each side.

5. At each x-coordinate, the visible skyline height is max(leftCurrentHeight, rightCurrentHeight).

6. Remove redundant points while constructing the result.

Grading Rubric

Criterion	Points	Description
Input reading and exact output format	10	Program works with the provided test driver and prints the skyline in the required format.
MergeSort implementation	15	Correct divide-and-conquer sorting by

		left coordinate, with correct tie-breaking.
Recursive skyline construction	20	Correct base case and recursive division of the building array.
Skyline merge operation	30	Correctly merges two skylines, handles overlaps, equal x-coordinates, and height updates.
Edge cases	10	Handles non-overlapping buildings, nested buildings, same starting coordinate, same ending coordinate, and redundant points.
Time complexity and scalability	10	Runs in Theta(n log n) and works for the 100,000-building input file.
Code quality and restrictions	5	Clean, readable code; no method/file name changes; no forbidden built-in sort or non-divide-and-conquer solution.

Submission Rules

- You must upload only the specified Java files until **Sunday, May 15, 2026, at 23.59 to ESTUOYS.**
- Never change the method names and Java file names.
- Never change Building.java, SkylinePoint.java, and TestSkyline.java.
- Grouping is not allowed in this homework. Please obey the ethical rules.
- The submitted homework may be checked with a plagiarism control tool. Therefore, you should not submit work from other people or sources as homework.

How to Compile and Run

```
javac *.java
java TestSkyline inputs/small.input
```

Note for students. Before submitting your files, test your code with all input files. Your code must finish within a reasonable time for test_100000.input.